07강

프로그래밍 언어론

변수와 바인딩

컴퓨터과학과 이관용 교수

학습목차



1. 변수의 개요
2. 바인딩
3. 변수의 바인딩

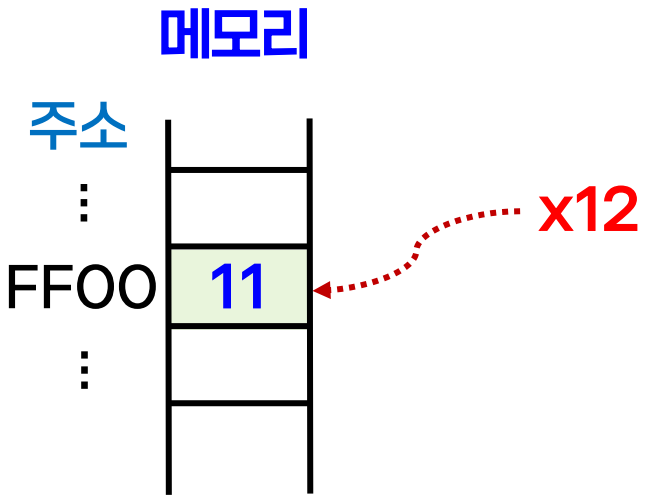
01

변수의 개요

변수 variable

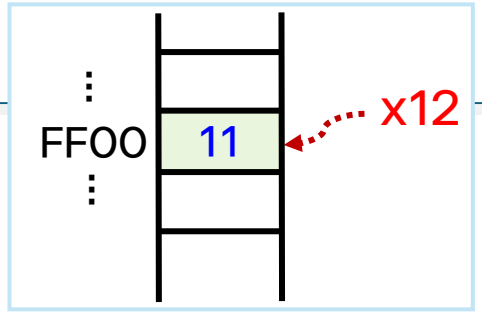
- 프로그램에서 처리할 데이터를 저장·관리할 수 있도록 컴퓨터 하드웨어의 메모리를 추상화한 것

```
int x12;
x12 = 1 + 5 * 2;
if x12 > 10 then ...
```



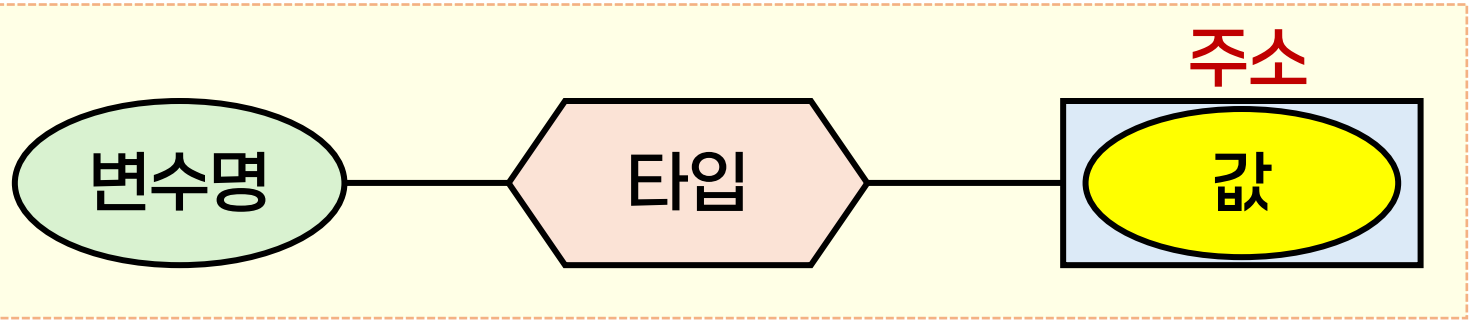
- 데이터를 저장하는 메모리 공간에 대한 상대적 이름

변수의 속성

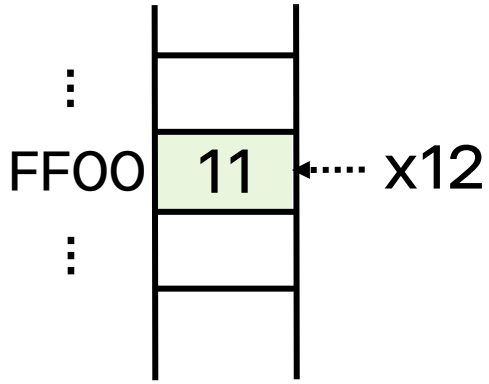


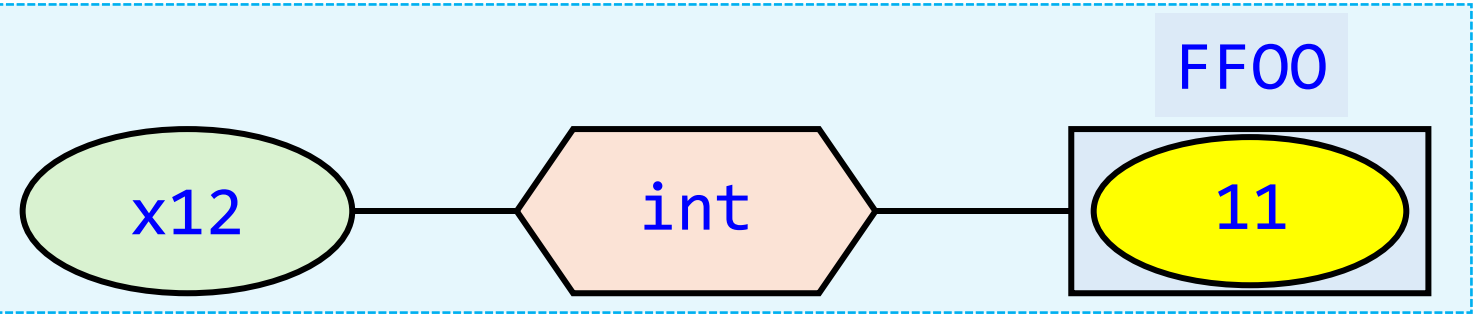
- 변수명 name → 변수의 이름. 식별자 identifier
- 타입 type → 변수에 저장할 수 있는 데이터 집합의 종류. 자료형
- 주소 address → 변수가 사용하는 메모리의 위치. "참조 reference"
- 값 value → 변수에 저장된 데이터 → 수행 시간 동안 변경 가능
- 수명 lifetime → 변수가 메모리를 할당받고 있는 기간
- 영역 scope → 프로그램에서 변수를 사용할 수 있는 범위

배런 Barron의 변수 표기법




```
int x12;
x12 = 1 + 5 * 2;
```



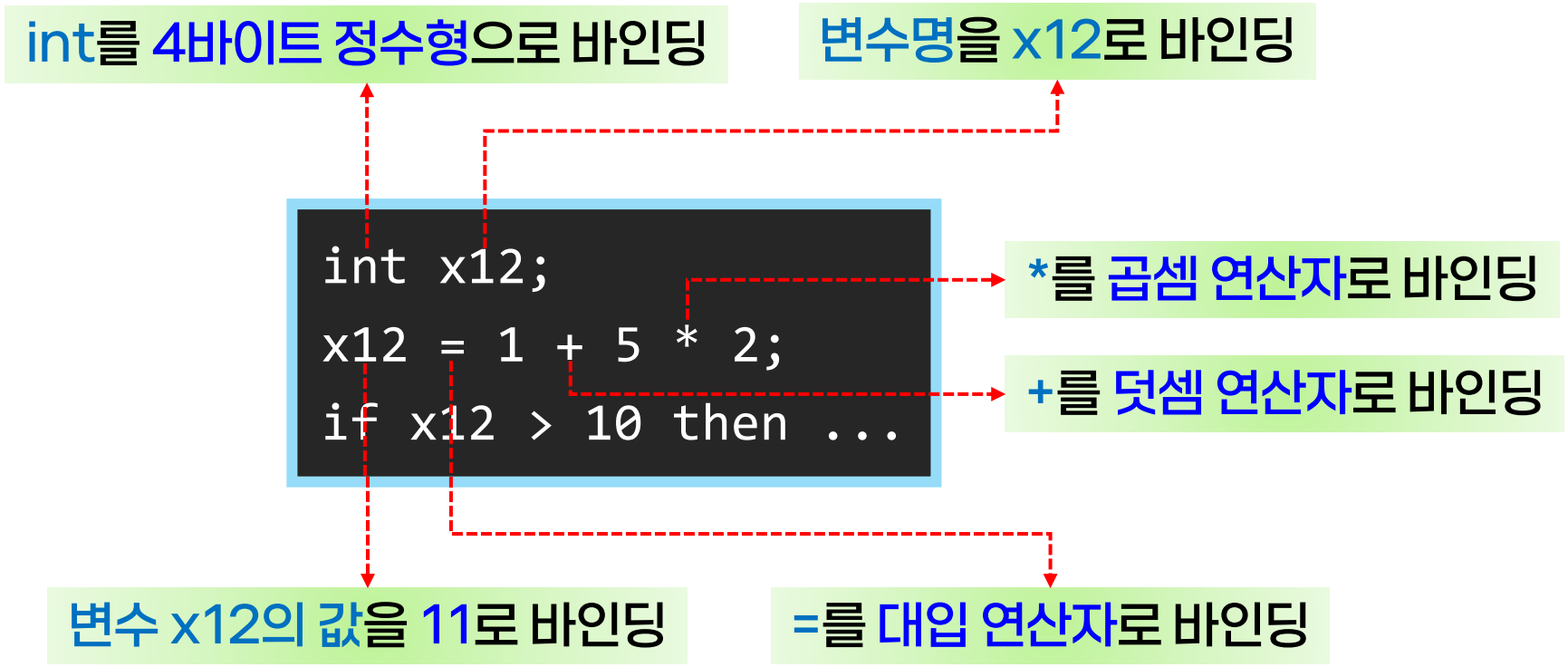


02

바인딩

바인딩 binding

- 언어 구성 요소의 속성이 구체적으로 결정되는 것

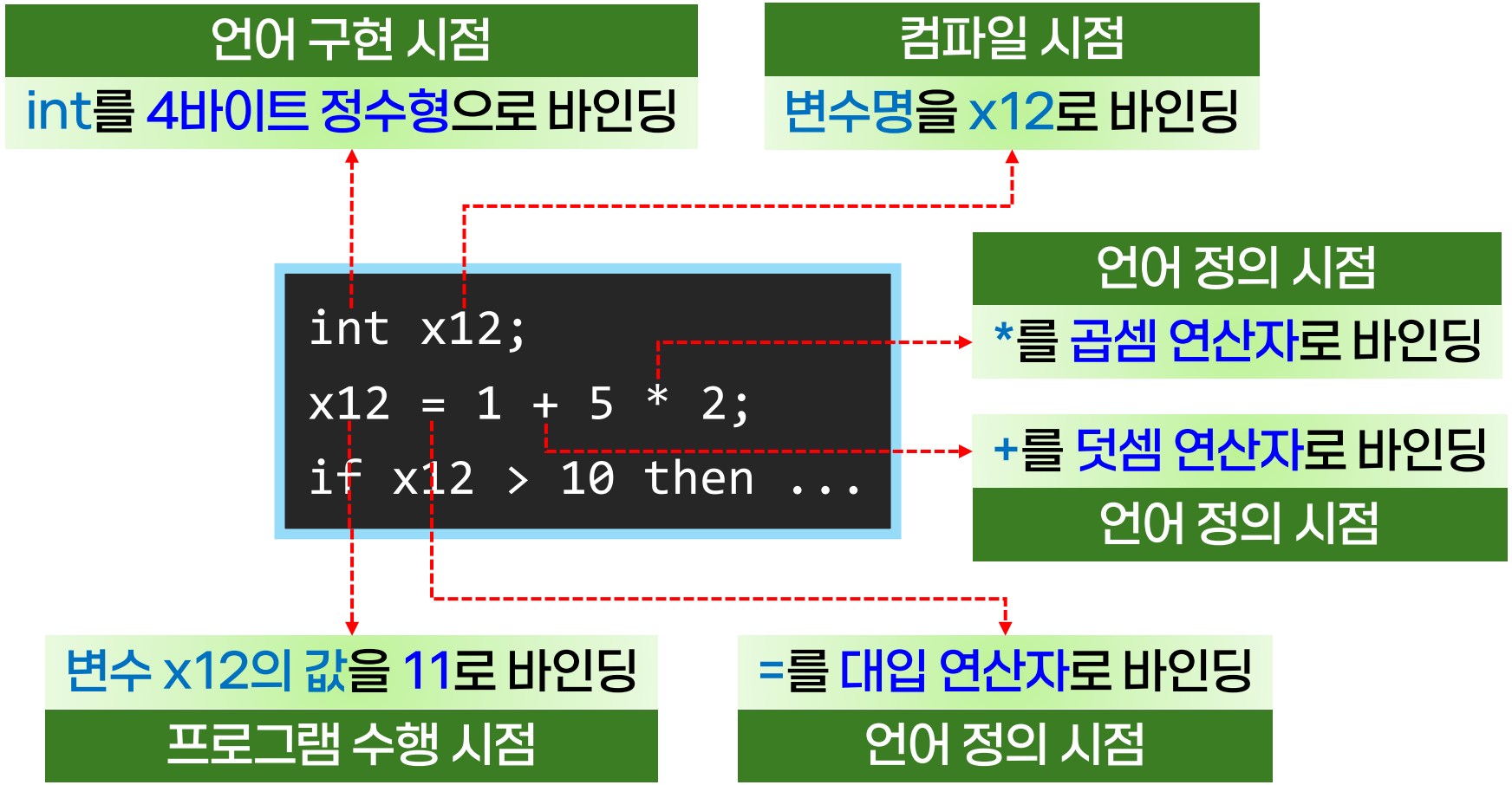


바인딩 시각 binding time

- 바인딩이 일어나는 시점에 따른 구분
  - 언어 정의 시점
  - 언어 구현 시점
  - 컴파일 시점
  - 링크 시점
  - 로드 시점
  - 프로그램 수행 시점

바인딩 시각

- 언어의 구성 요소 혹은 속성에 따라 달라짐



바인딩 시각에 따른 대표적인 예

바인딩 시각	예시
언어 정의 시점	연산자의 의미, 수의 의미 등
언어 구현 시점	타입의 자릿수, 수의 표기법 등
컴파일 시점	변수의 타입 등
링크 시점	라이브러리 함수의 내용 등
로드 시점	변수의 주소 등
프로그램 수행 시점	변수의 값 등

바인딩 시각의 단순화된 구분

정적 바인딩

- 프로그램 수행 시점에 바인딩의 변화가 없는 경우

동적 바인딩

- 프로그램 수행 시점에 바인딩의 변화가 있는 경우

정적 바인딩 static binding

- 프로그램 수행 시점에 바인딩의 변화가 없는 경우
  - 언어 정의 시점, 언어 구현 시점, 컴파일 시점, 링크 시점, 로드 시점의 바인딩을 포함
  - 프로그램 수행시간의 효율성이 높음
  - 컴파일 방법에 적합

동적 바인딩 dynamic binding

- 프로그램 수행 시점에 바인딩 되거나 바인딩이 해지되는 경우
  - 실제 바인딩 시각이 프로그램 수행 시점인 경우만 해당
  - 유연한 프로그래밍이 가능
  - 인터프리터 방법에 적합

03

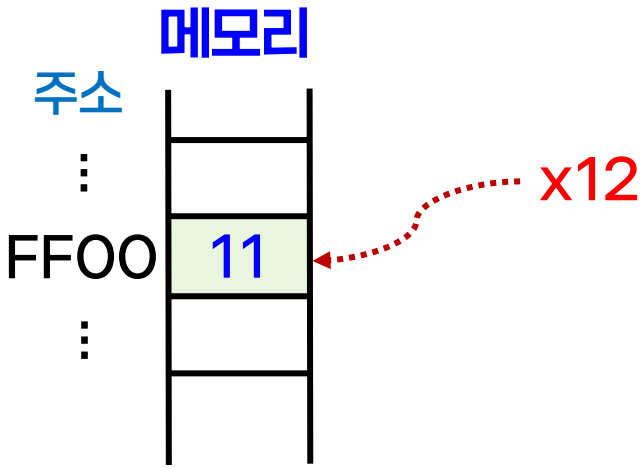
변수의 바인딩

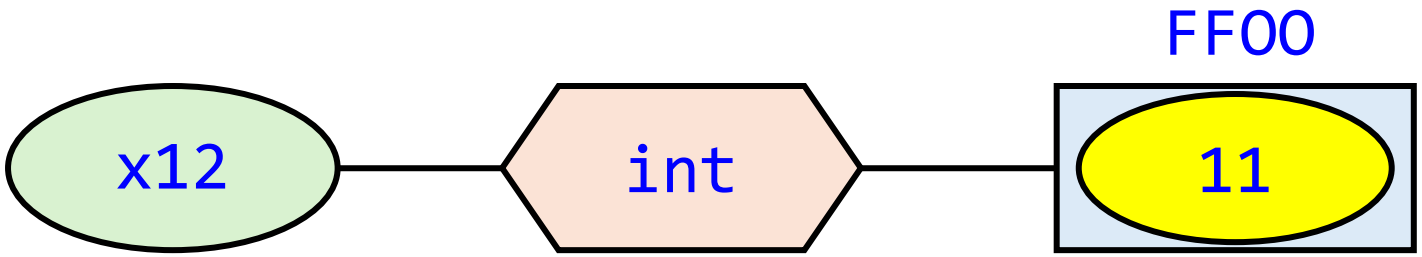
변수의 바인딩

- 변수의 속성이 구체적으로 결정되는 것
  - 프로그램 수행 중에 변수가 사용되기 위해서는 변수의 속성이 바인딩되어야 함
    - 변수의 속성 → 변수명, 타입, 주소, 값

변수 바인딩의 예

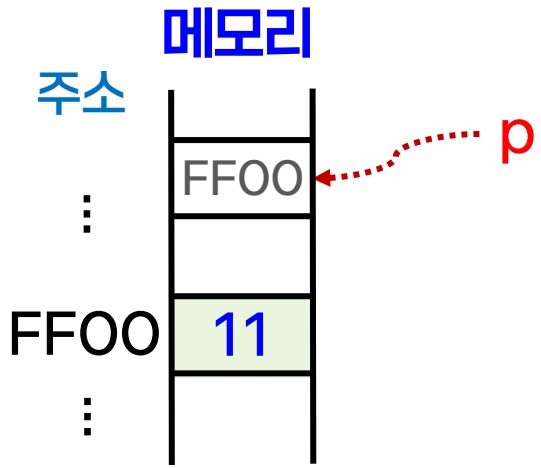
```
int x12;
x12 = 1 + 5 * 2;
if x12 > 10 then ...
```

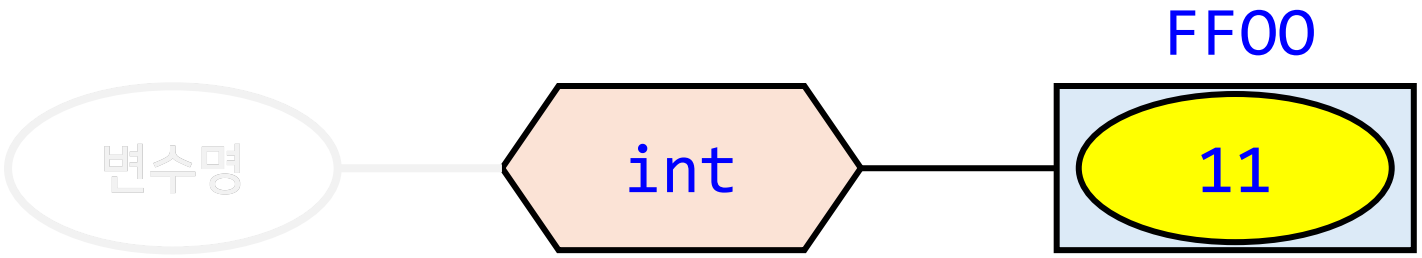




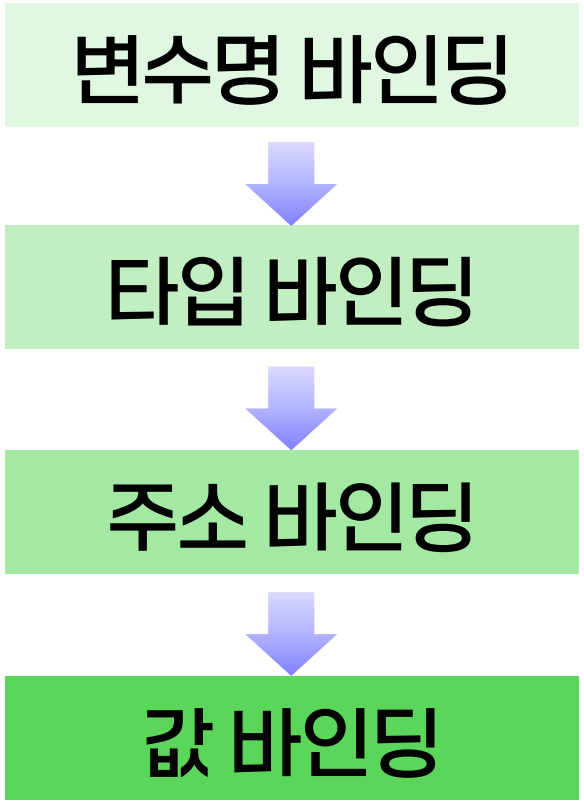
변수명에 대한 바인딩 없이 사용되는 경우

```
int *p = new int;
*p = 1 + 5 * 2;
if *p > 10 then ...
```





일반적인 변수의 속성별 바인딩 순서



(여러 번 발생할 수 있음)

변수명 바인딩

바인딩 방법

명시적 선언 explicit declaration

- 선언문에 명시된 이름으로 변수명을 바인딩

```
int x12;
```

묵시적 선언 implicit declaration

- 선언문 없음
- 대입문 등에 처음으로 사용된 이름으로 변수명을 바인딩

```
k = 123;
```

타입 바인딩

바인딩 방법

명시적 선언

- 선언문에 명시된 타입으로 변수의 타입을 바인딩

```
int x12;
```

묵시적 선언

- 변수명이나 대입할 값으로부터 정해지는 타입으로 바인딩
  - Fortran → 변수명이 i~n으로 시작하면 정수형으로 바인딩

```
i2 = …
```

  - Python → 대입할 값으로부터 타입을 바인딩

```
k = 123;
k = 'a';
k = 1.23;
```

타입 바인딩

타입 바인딩 시각

정적 바인딩

- 컴파일 시점에 구문 분석을 통해 타입을 판단
- 명시적 선언과 묵시적 선언 모두 가능

동적 바인딩

- 변수의 타입을 고정하지 않고, 대입할 값에 맞춰 계속 변화 시킴
- APL, SNOBOL 4, Python 등

```
k = 123;
k = 'abc';
```

정수형(int) 바인딩

문자열형(str) 바인딩

주소 바인딩

- 변수가 사용할 메모리가 할당되어
  변수의 주소가 그 메모리의 주소로 바인딩되는 것
  - 할당 allocation → 가용한 메모리 중에서 필요한 만큼의 공간을 변수에 배정하는 것
  - 해제 deallocation → 할당된 메모리를 변수로부터 회수해서 가용 메모리로 돌려주는 것

- 변수의 수명 lifetime/extent
  - 변수가 메모리를 할당받고 있는 기간
  - 변수의 주소가 바인딩된 기간

주소 바인딩

바인딩 방법

자동 할당

- 선언(명시적/묵시적)을 통해 정해진 변수의 타입에 맞추어 필요한 메모리를 할당하는 방법

```
int x;
double y;
```

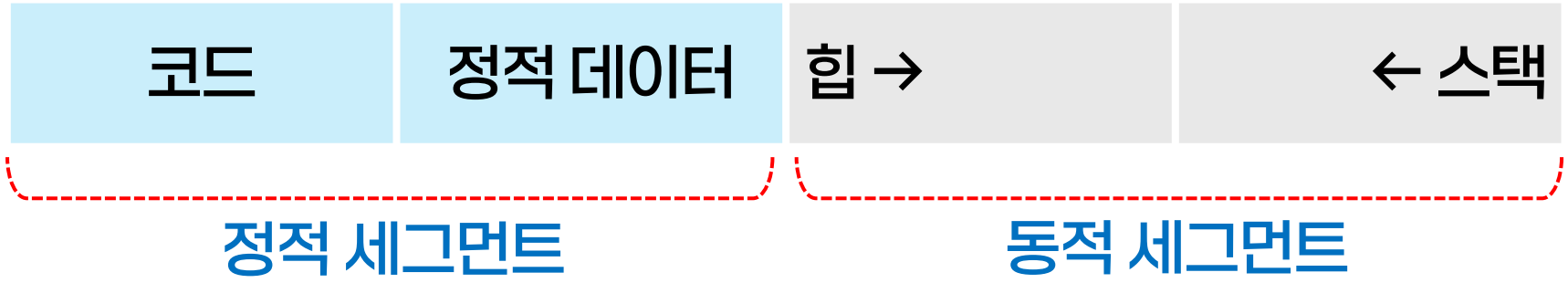
수동 할당

- 프로그래머가 지정한 크기 만큼의 메모리를 할당하는 방법

```
int *p = (int*) malloc(3*sizeof(int));
double *d = new double[4];
```

주소 바인딩

- 주소 바인딩을 위한 메모리 구조



- 자동 할당 → 상황/언어에 따라 정적 또는 동적 세그먼트를 활용해서 메모리 할당
- 수동 할당 → 동적 세그먼트의 힙을 활용

주소 바인딩

바인딩 시각

정적 바인딩

- 로드 시점에 정적 세그먼트의 주소를 바인딩하여 프로그램의 수행이 끝날 때까지 바인딩을 유지

→ 정적 변수
static variable

동적 바인딩

- 프로그램 수행 중 변수가 사용되는 시점에 동적 세그먼트의 주소를 바인딩

→ 동적 변수
dynamic variable

주소 바인딩

- 정적 변수
  - 정적 바인딩을 하는 변수
  - 수명 → 프로그램 수행 전체
  - 초기 Fortran의 모든 변수

주소 바인딩

동적 변수

- 동적 바인딩을 하는 변수
- 동적 세그먼트의 사용 영역에 따라

  → 스택 동적 변수, 힙 동적 변수

- 스택 동적 변수
  - 스택에서 메모리를 할당받는 변수
  - 자동 할당을 이용하는 변수 중 정적 변수가 아닌 경우
  - 필요한 시점에 자동으로 할당/해제됨

주소 바인딩

동적 변수

힙 동적 변수

- 동적 세그먼트 중 힙에서 메모리를 할당받는 변수
- 수동 할당을 이용한 변수

```
double *d = new double[4];      //수동 할당
…
delete[] d;                     //수동 해제
```

할당 malloc(), 해제 free() in C

- 동적 타입 바인딩을 하는 언어(APL, SNOBOL 4, Python 등)의 변수

정리하기

변수의 개요

- 정의 → 데이터를 저장하거나 나중에 사용할 수 있도록 메모리를 추상화한 것
- 속성(변수명, 타입, 주소, 값, 수명, 영역), 배런의 변수 표기법

바인딩

- 바인딩 → 언어 구성 요소의 속성이 구체적으로 결정되는 것
- 바인딩 시각의 구분 → 바인딩이 일어나는 시점, 정적 바인딩 vs 동적 바인딩

변수의 바인딩

- 변수의 바인딩 → 변수의 속성이 구체적으로 결정되는 것 → 변수명, 타입, 주소, 값 바인딩 순서
- 변수명 바인딩 → 명시적 선언 vs 묵시적 선언
- 타입 바인딩 → 명시적 선언 vs 묵시적 선언, 정적 바인딩 vs 동적 바인딩
- 주소 바인딩 → 할당/해제, 수명, 자동 할당 vs 수동 할당, 정적 세그먼트 vs 동적 세그먼트, 정적 바인딩 vs 동적 바인딩, 정적 변수 vs 동적 변수, 스택 동적 변수 vs 힙 동적 변수

08강

다음시간안내 ▶▶▶

변수의 영역